

FITBUDDY

AI FITNESS PLAN GENERATOR

Professional Project Documentation

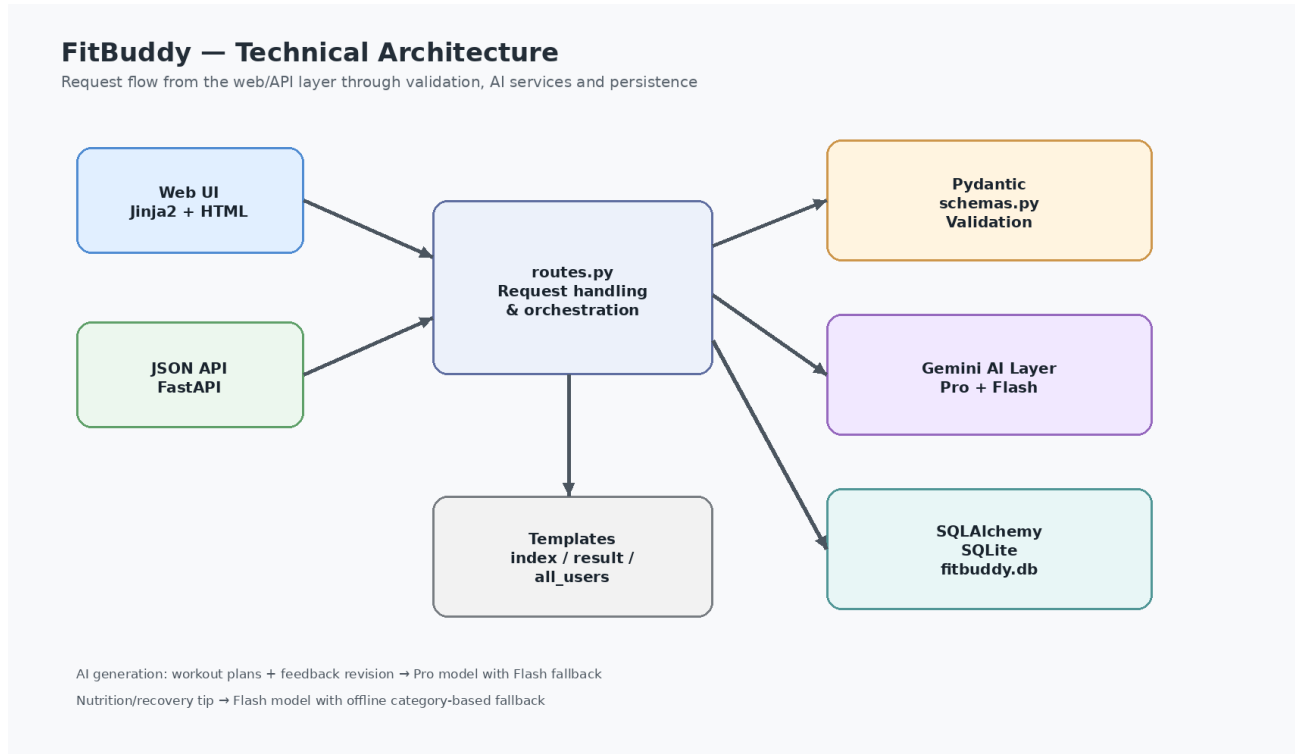


Figure 1. Technical architecture of the FitBuddy application.

Technology: Python • FastAPI • Google Gemini • SQLAlchemy • SQLite • Jinja2 • Pydantic • Pytest

Contents

1. Project Description
2. Objectives and Scenarios
3. Technical Architecture
4. Prerequisites and Environment Setup
5. Project Structure
6. Milestone 1 — Model Selection and Architecture
7. Milestone 2 — Core Functionality Development
8. Milestone 3 — API and Route Development
9. Milestone 4 — Frontend Development
10. Milestone 5 — Deployment and Testing
11. API Reference
12. Database Design
13. Security, Reliability and Error Handling
14. Screenshots and Implementation Evidence
15. Conclusion

1. Project Description

FitBuddy is a FastAPI-based web application that uses Google Gemini models to generate personalized 7-day workout plans and concise nutrition or recovery tips. The current implementation accepts user profile information such as name, user ID, age, weight, fitness goal and workout intensity, validates the data, sends an appropriate prompt to Gemini, stores the resulting plan in SQLite, and renders the result through Jinja2 templates.

A feedback loop allows an existing plan to be revised. The current plan is retrieved from the database, combined with the user's feedback, and sent back through the Gemini generation layer. The application also exposes JSON endpoints for programmatic use and an administrative page for viewing stored users and plans.

The uploaded reference documentation follows a milestone-based project-report structure; this document keeps that organization while updating the technical content to match the supplied project archive.

2. Objectives and Scenarios

2.1 Objectives

- Generate structured, personalized 7-day workout plans from a user's goal, intensity, age and weight.
- Provide a concise nutrition or recovery tip using a fast Gemini model, with a built-in fallback when the AI service is unavailable.
- Allow users to revise an existing plan through natural-language feedback.
- Persist user profiles and original/updated plans in SQLite through SQLAlchemy.
- Provide both browser-based HTML workflows and JSON API endpoints.
- Provide automated tests using a fake Gemini client so the test suite does not require an API key or network access.

2.2 Main Scenarios

Scenario	Description
Scenario 1 — Plan generation	User enters profile data and receives a personalized 7-day plan.
Scenario 2 — Plan revision	User submits feedback such as adding cardio or changing difficulty; the current plan is revised by Gemini.
Scenario 3 — Nutrition tip	The system generates a short nutrition/recovery tip for the selected goal; an offline category-based tip is used if Gemini fails.
Scenario 4 — Administration	A local admin page lists registered users, original plans, updated plans and provides deletion actions.
Scenario 5 — API integration	External clients can call JSON endpoints for workout generation, nutrition tips, plan creation, plan updates

and user deletion.

3. Technical Architecture

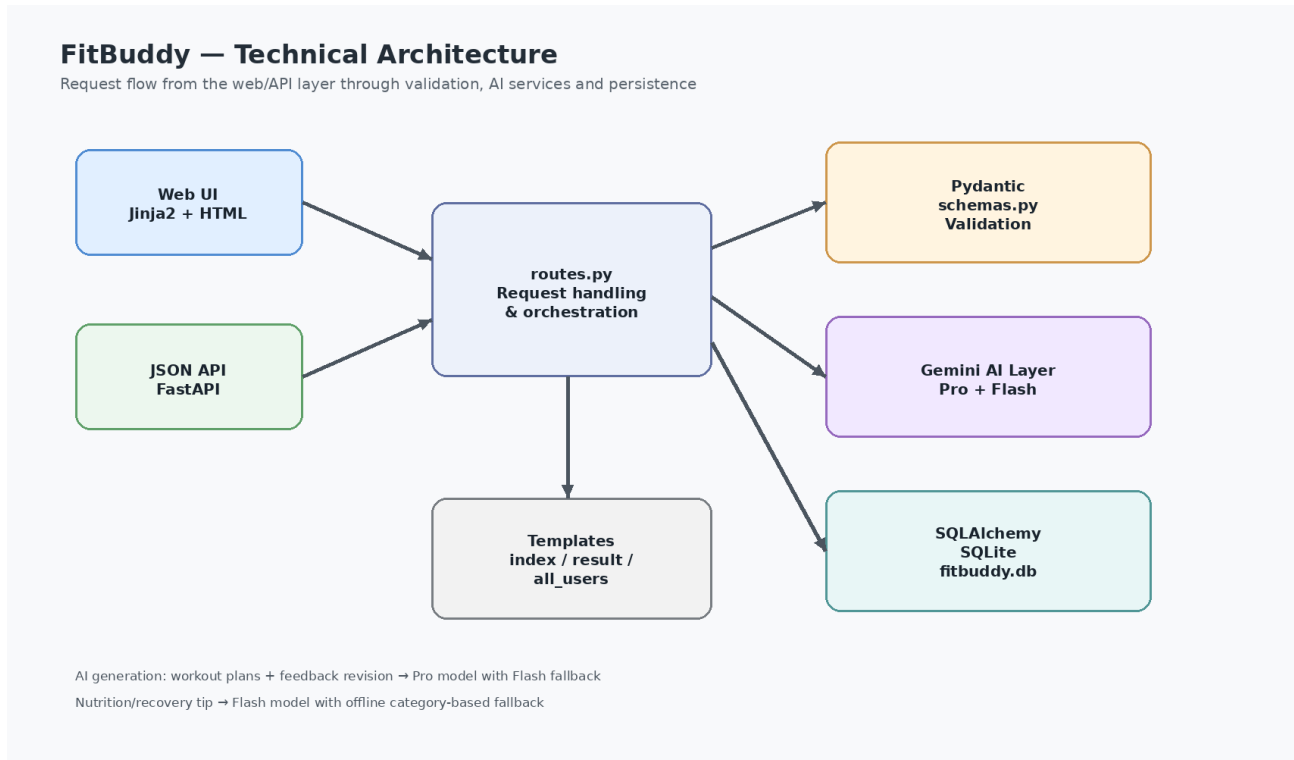


Figure 2. Application architecture derived from the supplied source code.

3.1 Architecture Layers

Layer	Technology	Responsibility
Presentation	HTML + Jinja2 + static assets	Collect input, display generated plans, feedback form and user table.
Web/API	FastAPI	Routes requests, validates inputs and orchestrates services.
Validation	Pydantic	Enforces ranges, lengths and allowed intensity values.
AI	Google Gen AI SDK / Gemini	Generates workout plans, nutrition tips and feedback-based revisions.
Persistence	SQLAlchemy + SQLite	Stores users and original/updated workout plans.
Testing	Pytest + fake Gemini	Exercises web and API flows, validation and AI failure paths.

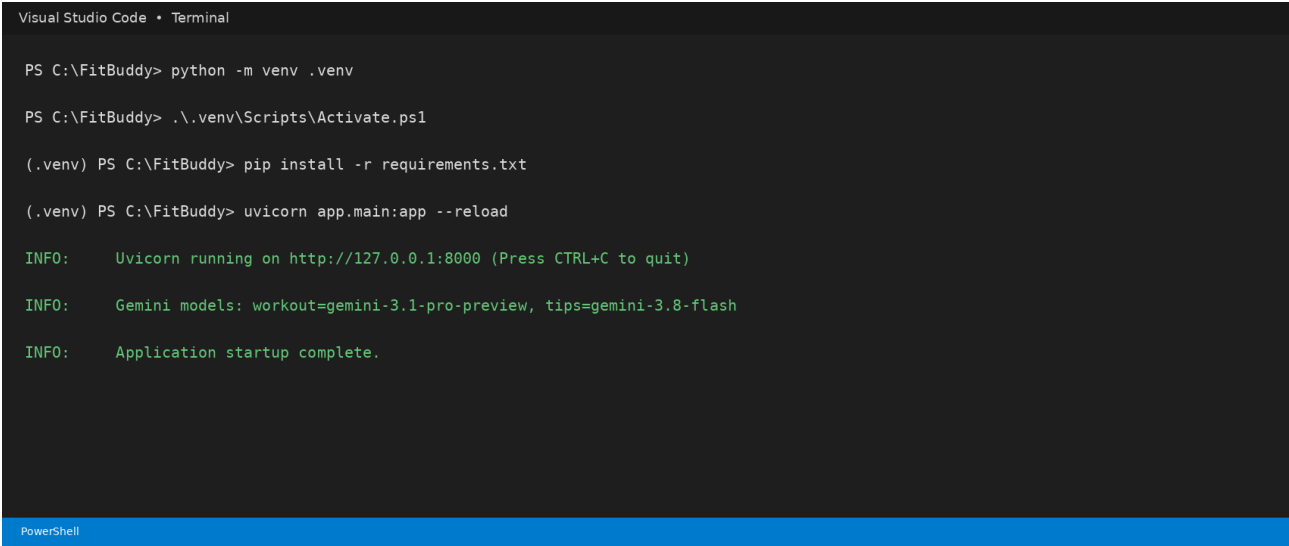
The architecture is modular: `routes.py` coordinates the request flow, while AI prompting, persistence, validation and configuration are separated into dedicated modules. This reduces coupling and makes the application easier to test and maintain.

4. Prerequisites and Environment Setup

- Python 3.10 or newer
- Visual Studio Code with the Python extension
- Google AI Studio / Gemini API key
- Basic Python, FastAPI, HTML/Jinja2 and SQLAlchemy knowledge
- Git for version control (recommended)

4.1 Installation

16. Open the FitBuddy project folder in Visual Studio Code.
17. Create a virtual environment: `python -m venv .venv`
18. Activate it on Windows: `.venv\Scripts\Activate.ps1`
19. Install dependencies: `pip install -r requirements.txt`
20. Copy `.env.example` to `.env` and set `GEMINI_API_KEY`.
21. Start the server: `uvicorn app.main:app --reload`
22. Open <http://127.0.0.1:8000> and the interactive API documentation at `/docs`.



```
Visual Studio Code • Terminal

PS C:\FitBuddy> python -m venv .venv

PS C:\FitBuddy> .venv\Scripts\Activate.ps1

(.venv) PS C:\FitBuddy> pip install -r requirements.txt

(.venv) PS C:\FitBuddy> uvicorn app.main:app --reload

INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Gemini models: workout=gemini-3.1-pro-preview, tips=gemini-3.8-flash
INFO:      Application startup complete.
```

Figure 3. VS Code-style terminal showing the documented local setup and Uvicorn launch sequence.

4.2 Configuration

Variable	Default / Requirement	Purpose
GEMINI_API_KEY	Required	Authentication for Google Gemini.
GEMINI_PRO_MODEL	gemini-3.1-pro-preview	Workout plans and feedback revisions.
GEMINI_FLASH_MODEL	gemini-3.8-flash	Nutrition tips and fallback model.
GEMINI_TIMEOUT_SECONDS	90	Gemini request timeout in seconds.
DATABASE_URL	sqlite:///fitbuddy.db	SQLAlchemy database connection.

Model names are configurable because Gemini model availability changes over time. The project README explicitly recommends updating these environment variables if a configured model is retired or unavailable.

5. Project Structure

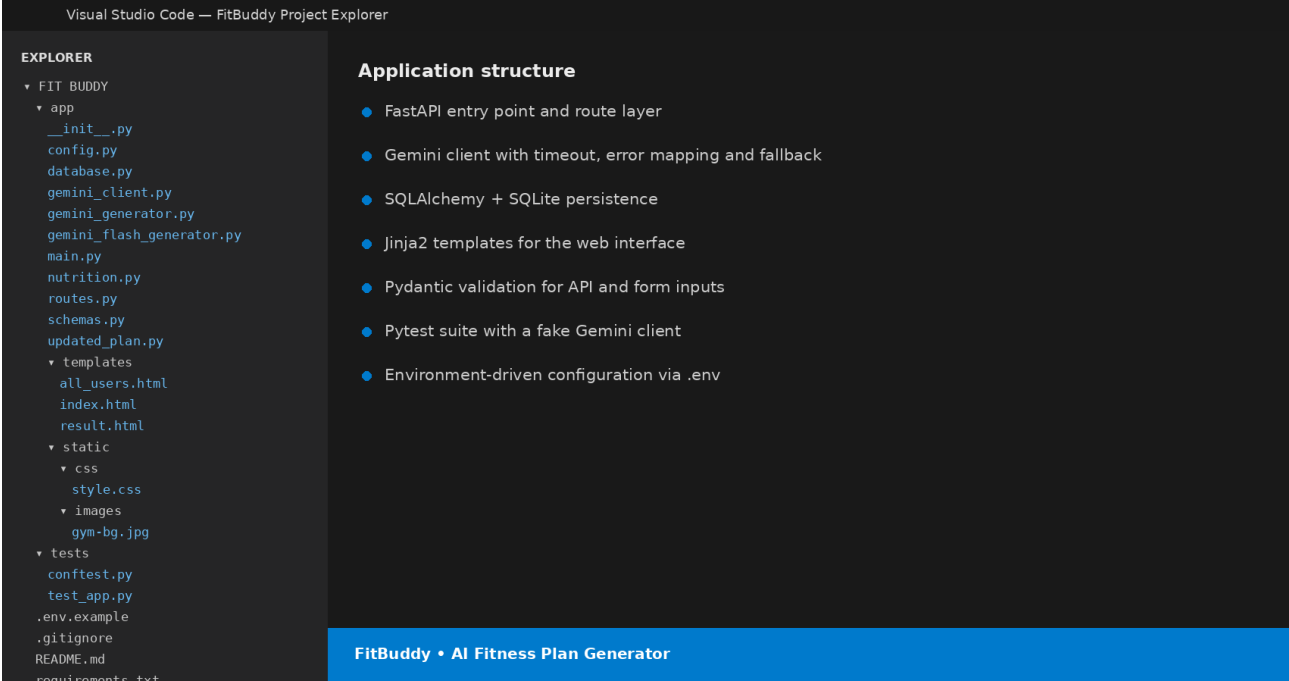


Figure 4. Project structure represented in a VS Code Explorer-style view.

Path	Purpose
app/main.py	FastAPI application entry point, startup lifecycle and /health endpoint.
app/routes.py	JSON API routes and HTML page routes.
app/config.py	Environment-driven configuration and Gemini model settings.
app/database.py	SQLAlchemy models and database helper functions.
app/schemas.py	Pydantic request models and input validation.
app/gemini_client.py	Central Gemini SDK wrapper, error mapping, timeout and fallback.
app/gemini_generator.py	Workout prompt construction and plan generation.
app/gemini_flash_generator.py	Short nutrition/recovery tip generation.
app/updated_plan.py	Feedback-based workout plan revision.
app/nutrition.py	Nutrition helper and offline fallback tips.
app/templates/	Jinja2 pages: index, result and all_users.
app/static/	CSS and fitness background image assets.
tests/	Pytest fixtures and end-to-end application tests.
requirements.txt	Runtime and testing dependencies.
fitbuddy.db	SQLite persistence file.

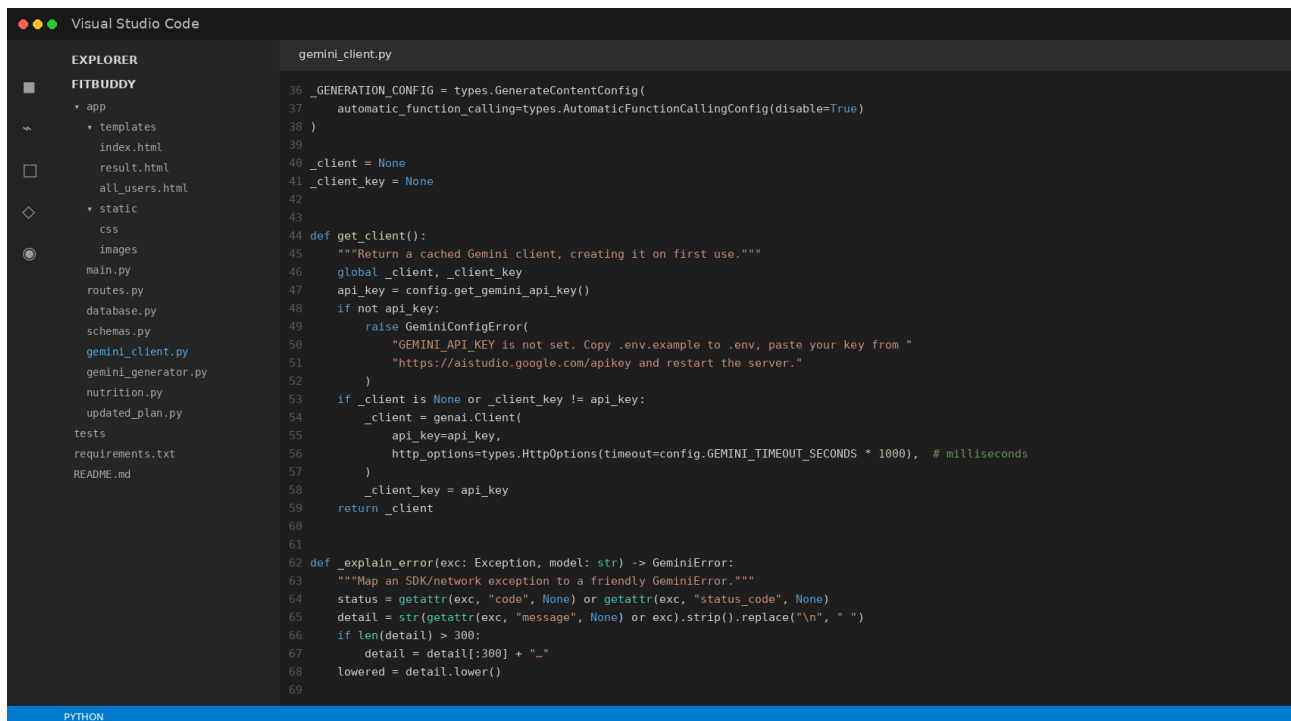
6. Milestone 1 — Model Selection and Architecture

6.1 Model Selection

The supplied reference document describes an earlier evaluation of Hugging Face models and a previous Gemini model selection. The current project archive supersedes those model names: its configuration defaults are gemini-3.1-pro-preview for the Pro role and gemini-3.8-flash for the Flash role. The model names are not hard-coded into the business logic; they are read from environment variables.

6.2 Why the AI layer is centralized

- All Gemini calls pass through `generate_text()`, which creates the client lazily and avoids requiring a key just to start the application.
- SDK/network errors are converted into `GeminiError` messages that are suitable for application handling.
- Empty Gemini responses are treated as errors rather than silently becoming empty plans.
- Workout generation and feedback revision use `generate_with_fallback()`, allowing a non-fatal Pro-model failure to retry through the Flash model.



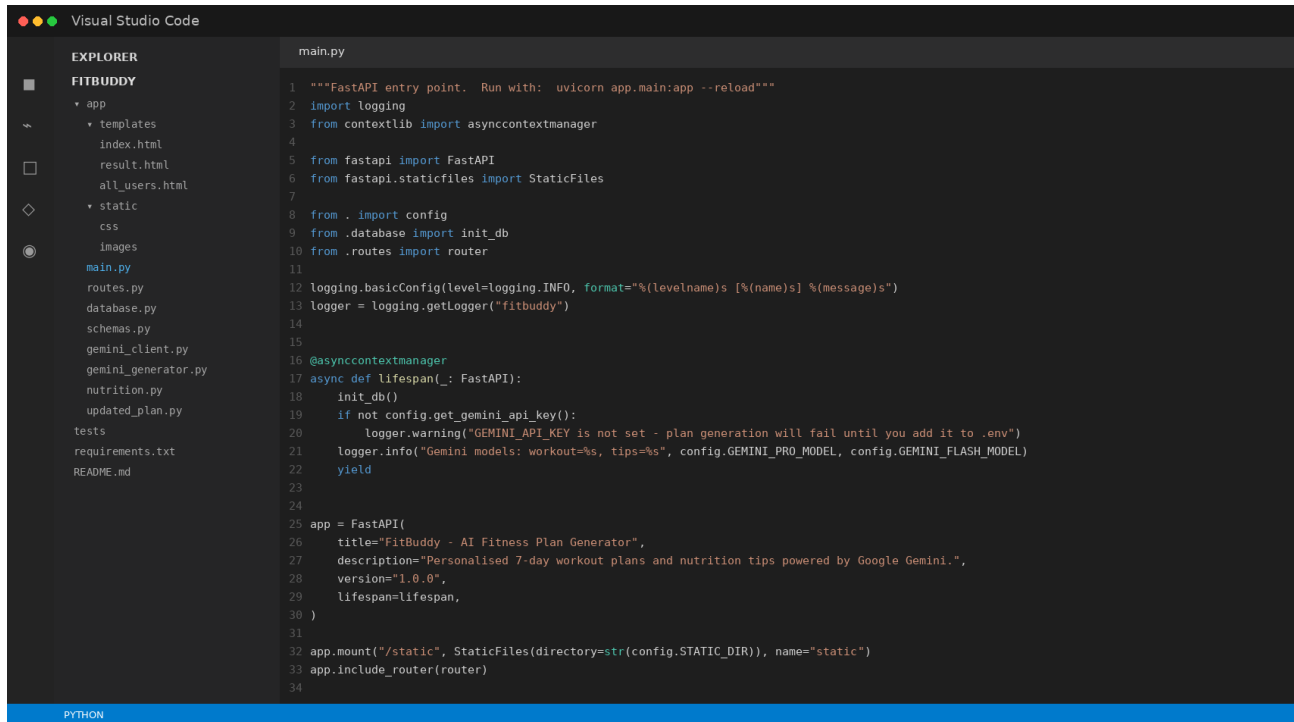
```
36 _GENERATION_CONFIG = types.GenerateContentConfig(
37     automatic_function_calling=typesAutomaticFunctionCallingConfig(disable=True)
38 )
39
40 _client = None
41 _client_key = None
42
43
44 def get_client():
45     """Return a cached Gemini client, creating it on first use."""
46     global _client, _client_key
47     api_key = config.get_gemini_api_key()
48     if not api_key:
49         raise GeminiConfigError(
50             "GEMINI_API_KEY is not set. Copy .env.example to .env, paste your key from "
51             "https://aistudio.google.com/apikey and restart the server."
52         )
53     if _client is None or _client_key != api_key:
54         _client = genai.Client(
55             api_key=api_key,
56             http_options=types.HttpOptions(timeout=config.GEMINI_TIMEOUT_SECONDS * 1000), # milliseconds
57         )
58         _client_key = api_key
59     return _client
60
61
62 def explain_error(exc: Exception, model: str) -> GeminiError:
63     """Map an SDK/network exception to a friendly GeminiError."""
64     status = getattr(exc, "code", None) or getattr(exc, "status_code", None)
65     detail = str(getattr(exc, "message", None) or exc).strip().replace("\n", " ")
66     if len(detail) > 300:
67         detail = detail[:300] + "..."
68     lowered = detail.lower()
69
```

Figure 5. Gemini client implementation in a VS Code-style view, showing the centralized AI service layer.

7. Milestone 2 — Core Functionality Development

7.1 Workout Plan Generation

`generate_workout_gemini()` builds a structured prompt containing the user goal, intensity and optional age/weight. The prompt asks Gemini for seven days, with warm-up, main workout, cooldown/recovery guidance and important notes. The generated text is returned to the route layer and displayed in the result template.



```
1 """FastAPI entry point. Run with: uvicorn app.main:app --reload"""
2 import logging
3 from contextlib import asynccontextmanager
4
5 from fastapi import FastAPI
6 from fastapi.staticfiles import StaticFiles
7
8 from . import config
9 from .database import init_db
10 from .routes import router
11
12 logging.basicConfig(level=logging.INFO, format="%(levelname)s %(name)s %(message)s")
13 logger = logging.getLogger("fitbuddy")
14
15
16 @asynccontextmanager
17 async def lifespan(_ : FastAPI):
18     init_db()
19     if not config.get_gemini_api_key():
20         logger.warning("GEMINI_API_KEY is not set - plan generation will fail until you add it to .env")
21     logger.info("Gemini models: workout=%s, tips=%s", config.GEMINI_PRO_MODEL, config.GEMINI_FLASH_MODEL)
22     yield
23
24
25 app = FastAPI(
26     title="FitBuddy - AI Fitness Plan Generator",
27     description="Personalised 7-day workout plans and nutrition tips powered by Google Gemini.",
28     version="1.0.0",
29     lifespan=lifespan,
30 )
31
32 app.mount("/static", StaticFiles(directory=str(config.STATIC_DIR)), name="static")
33 app.include_router(router)
34
```

Figure 6. FastAPI application startup and routing setup from `app/main.py`.

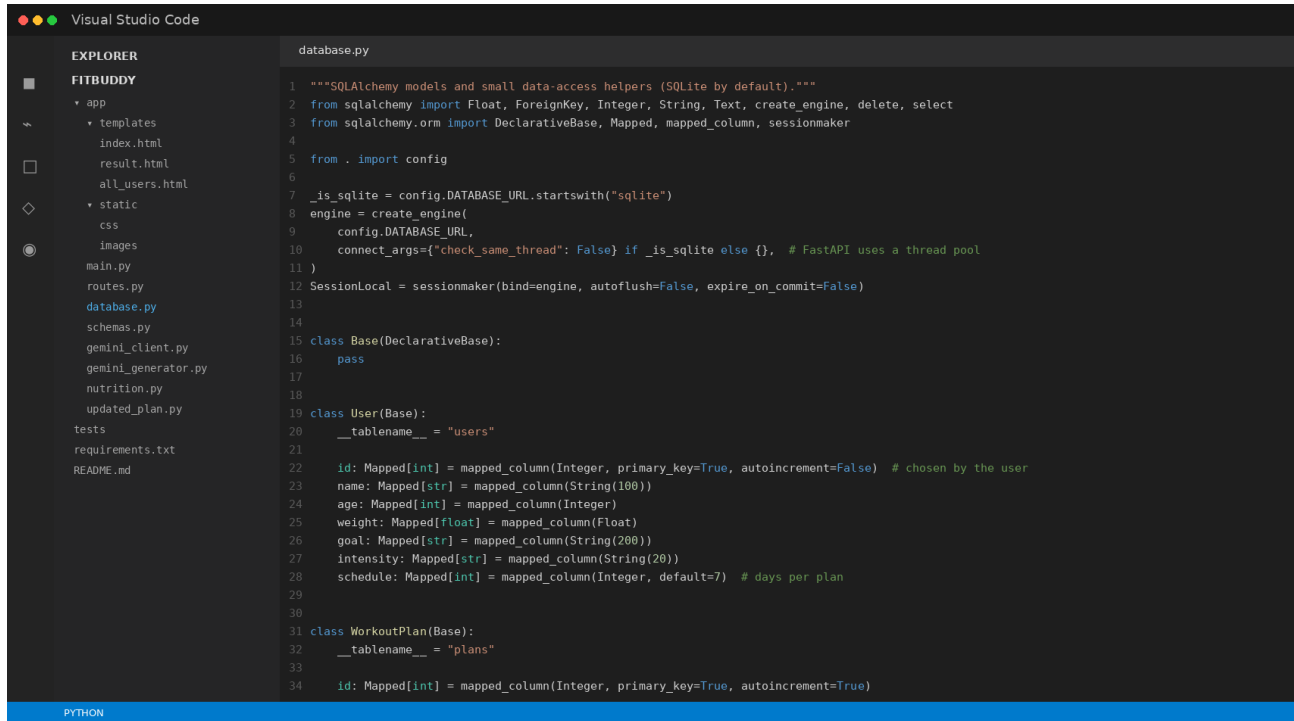
7.2 Nutrition and Recovery Tips

`get_nutrition_tip_with_source()` first attempts Gemini Flash. If a `GeminiError` occurs, the nutrition module categorizes the free-text goal using keywords and selects a built-in fallback tip for weight loss, muscle gain, flexibility or general wellness.

7.3 Feedback-Based Plan Updating

`update_workout_plan()` combines the current plan and user feedback into a revision prompt. The route then stores the returned text in `updated_plan` while preserving the original plan.

7.4 User and Plan Storage

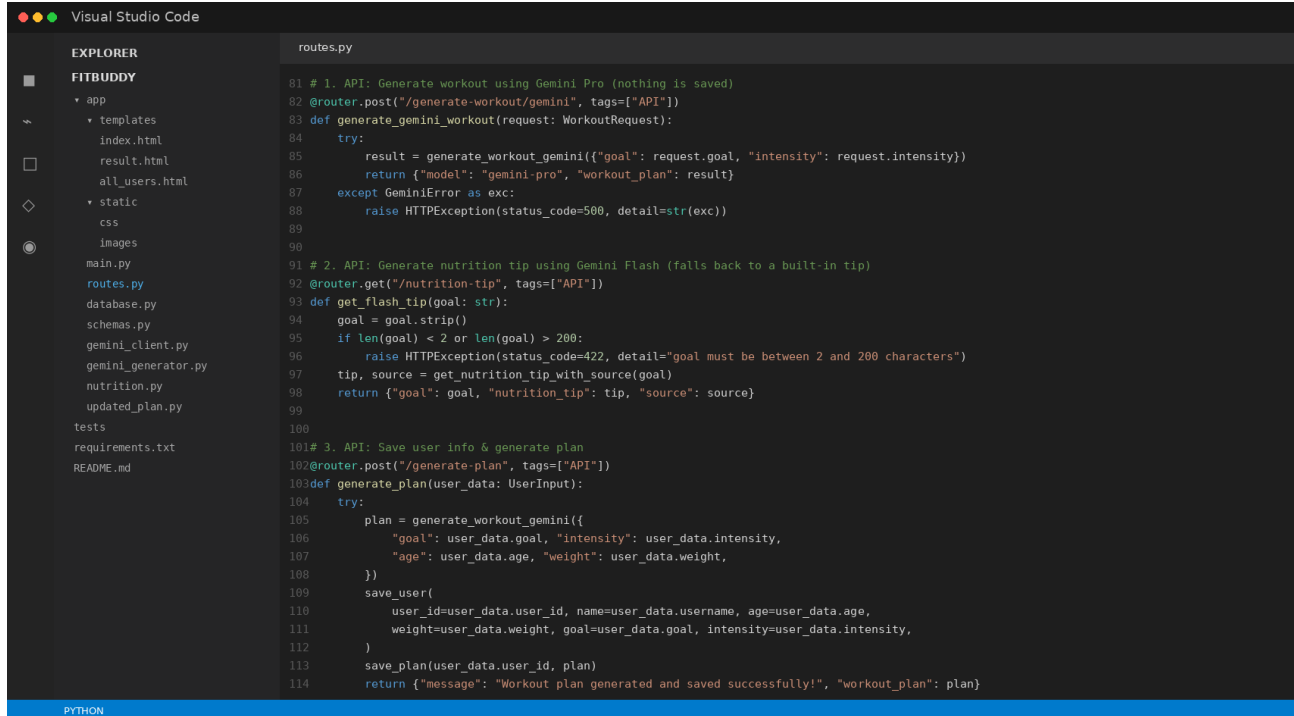


```
1 """SQLAlchemy models and small data-access helpers (SQLite by default)."""
2 from sqlalchemy import Float, ForeignKey, Integer, String, Text, create_engine, delete, select
3 from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, sessionmaker
4
5 from . import config
6
7 _is_sqlite = config.DATABASE_URL.startswith("sqlite")
8 engine = create_engine(
9     config.DATABASE_URL,
10     connect_args={"check_same_thread": False} if _is_sqlite else {}, # FastAPI uses a thread pool
11 )
12 SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)
13
14
15 class Base(DeclarativeBase):
16     pass
17
18
19 class User(Base):
20     __tablename__ = "users"
21
22     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=False) # chosen by the user
23     name: Mapped[str] = mapped_column(String(100))
24     age: Mapped[int] = mapped_column(Integer)
25     weight: Mapped[float] = mapped_column(Float)
26     goal: Mapped[str] = mapped_column(String(200))
27     intensity: Mapped[str] = mapped_column(String(20))
28     schedule: Mapped[int] = mapped_column(Integer, default=7) # days per plan
29
30
31 class WorkoutPlan(Base):
32     __tablename__ = "plans"
33
34     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
```

Figure 7. SQLAlchemy models and data-access helpers from app/database.py.

The User table stores the profile and workout intensity. The WorkoutPlan table stores original_plan and an optional updated_plan. Regenerating a user plan replaces the current original plan and clears a previous update, keeping one current plan per user.

8. Milestone 3 — API and Route Development



```
81 # 1. API: Generate workout using Gemini Pro (nothing is saved)
82 @router.post("/generate-workout/gemini", tags=["API"])
83 def generate_gemini_workout(request: WorkoutRequest):
84     try:
85         result = generate_workout_gemini({"goal": request.goal, "intensity": request.intensity})
86         return {"model": "gemini-pro", "workout_plan": result}
87     except GeminiError as exc:
88         raise HTTPException(status_code=500, detail=str(exc))
89
90
91 # 2. API: Generate nutrition tip using Gemini Flash (falls back to a built-in tip)
92 @router.get("/nutrition-tip", tags=["API"])
93 def get_flash_tip(goal: str):
94     goal = goal.strip()
95     if len(goal) < 2 or len(goal) > 200:
96         raise HTTPException(status_code=422, detail="goal must be between 2 and 200 characters")
97     tip, source = get_nutrition_tip_with_source(goal)
98     return {"goal": goal, "nutrition_tip": tip, "source": source}
99
100
101 # 3. API: Save user info & generate plan
102 @router.post("/generate-plan", tags=["API"])
103 def generate_plan(user_data: UserInput):
104     try:
105         plan = generate_workout_gemini({
106             "goal": user_data.goal, "intensity": user_data.intensity,
107             "age": user_data.age, "weight": user_data.weight,
108         })
109         save_user(
110             user_id=user_data.user_id, name=user_data.username, age=user_data.age,
111             weight=user_data.weight, goal=user_data.goal, intensity=user_data.intensity,
112         )
113         save_plan(user_data.user_id, plan)
114         return {"message": "Workout plan generated and saved successfully!", "workout_plan": plan}
```

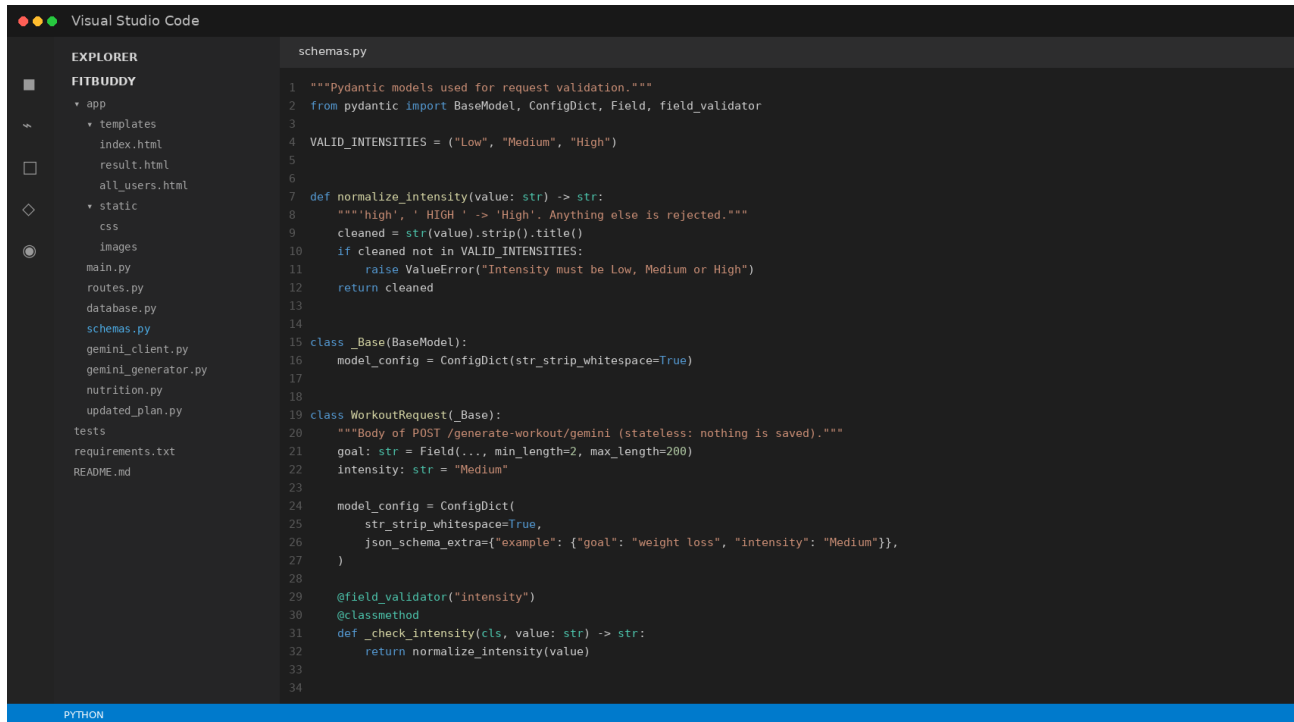
Figure 8. Core JSON API routes in app/routes.py.

8.1 Route Summary

Method	Endpoint	Purpose
GET	/	HTML input form.
POST	/generate	Submit web form and generate/store a plan.
POST	/submit-feedback	Revise a stored plan using feedback.
GET	/view-all-users	Local admin table of users and plans.
POST	/delete-user/{user_id}	Delete a user from the web admin page.
POST	/generate-workout/gemini	Stateless JSON workout generation.
GET	/nutrition-tip	JSON nutrition/recovery tip.
POST	/generate-plan	Create/save user and generated plan.
POST	/update-plan/{user_id}	Update the current plan through feedback.
DELETE	/users/{user_id}	Delete user and plans through JSON API.
GET	/health	Returns {"status":"ok"}.
GET	/docs	FastAPI interactive API documentation.

8.2 Validation

Pydantic models enforce required fields and sensible ranges. User IDs must be positive, ages are constrained to 10–100, weight must be greater than zero and no more than 500 kg, goals have a 2–200 character range, and feedback is limited to 3–1000 characters. Workout intensity is normalized to Low, Medium or High.



```
1 """Pydantic models used for request validation."""
2 from pydantic import BaseModel, ConfigDict, Field, field_validator
3
4 VALID_INTENSITIES = ("Low", "Medium", "High")
5
6
7 def normalize_intensity(value: str) -> str:
8     """'high', ' HIGH ' -> 'High'. Anything else is rejected."""
9     cleaned = str(value).strip().title()
10    if cleaned not in VALID_INTENSITIES:
11        raise ValueError("Intensity must be Low, Medium or High")
12    return cleaned
13
14
15 class _Base(BaseModel):
16     model_config = ConfigDict(str_strip_whitespace=True)
17
18
19 class WorkoutRequest(_Base):
20     """Body of POST /generate-workout/gemini (stateless: nothing is saved)."""
21     goal: str = Field(..., min_length=2, max_length=200)
22     intensity: str = "Medium"
23
24     model_config = ConfigDict(
25         str_strip_whitespace=True,
26         json_schema_extra={"example": {"goal": "weight loss", "intensity": "Medium"}},
27     )
28
29     @field_validator("intensity")
30     @classmethod
31     def _check_intensity(cls, value: str) -> str:
32         return normalize_intensity(value)
33
34
```

Figure 9. Pydantic validation models from app/schemas.py.

9. Milestone 4 — Frontend Development

9.1 Input Page

index.html collects name, user ID, age, weight, fitness goal and intensity. The form posts to /generate. JavaScript disables the submit button and changes its label while a potentially slow AI request is running.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>FitBuddy - AI Workout Generator</title>
7 <link rel="preconnect" href="https://fonts.googleapis.com">
8 <link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;500;700&display=swap" rel="stylesheet">
9 <link rel="stylesheet" href="{{ url_for('static', path='css/style.css') }}">
10 </head>
11 <body>
12 <div class="container">
13 <h1>FitBuddy - AI Workout Generator</h1>
14
15 {% if error %}
16 <div class="alert alert-error" role="alert">{{ error }}</div>
17 {% endif %}
18
19 <form action="/generate" method="post" id="plan-form">
20 <label for="username">Name:</label>
21 <input type="text" id="username" name="username" maxlength="100" required
22 value="{{ form.get('username', '') }}">
23
24 <label for="user_id">User ID:</label>
25 <input type="number" id="user_id" name="user_id" min="1" step="1" required
26 value="{{ form.get('user_id', '') }}">
27
28 <label for="age">Age:</label>
29 <input type="number" id="age" name="age" min="10" max="100" step="1" required
30 value="{{ form.get('age', '') }}">
31
32 <label for="weight">Weight (kg):</label>
33 <input type="number" id="weight" name="weight" min="1" max="500" step="0.1" required
34 value="{{ form.get('weight', '') }}">
```

Figure 10. Jinja2 input form source from index.html.

9.2 Result Page

result.html displays the user information, generated or updated plan, nutrition tip and a feedback form. The nutrition tip is carried as a hidden field so a feedback submission can reuse the current tip without making another nutrition-model call.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Your FitBuddy Workout Plan</title>
7 <link rel="preconnect" href="https://fonts.googleapis.com">
8 <link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;500;700&display=swap" rel="stylesheet">
9 <link rel="stylesheet" href="{{ url_for('static', path='css/style.css') }}">
10 </head>
11 <body>
12 <div class="container wide">
13 <h1>Your Personalized Workout Plan</h1>
14
15 {% if success_message %}
16 <div class="alert alert-success" role="status">{{ success_message }}</div>
17 {% endif %}
18 {% if error %}
19 <div class="alert alert-error" role="alert">{{ error }}</div>
20 {% endif %}
21
22 <h2>User Information</h2>
23 <div class="info">
24 <p><strong>Name:</strong> {{ username }}</p>
25 <p><strong>User ID:</strong> {{ user_id }}</p>
26 <p><strong>Age:</strong> {{ age }}</p>
27 <p><strong>Weight:</strong> {{ weight }} kg</p>
28 <p><strong>Goal:</strong> {{ goal }}</p>
29 <p><strong>Intensity:</strong> {{ intensity }}</p>
30 </div>
31
32 <h2>{{ plan_title | default('Workout Plan') }}</h2>
33 <pre class="plan">{{ workout_plan }}</pre>
34
```

Figure 11. Jinja2 result and feedback interface source from result.html.

9.3 Admin View

`all_users.html` renders all stored users and their original/updated plans in a table. The page also provides a delete action. Because the project explicitly documents this page as having no login, it should be kept local unless authentication and authorization are added.

10. Milestone 5 — Deployment and Testing

10.1 Local Deployment

- Create and activate `.venv`.
- Install `requirements.txt`.
- Configure `GEMINI_API_KEY` in `.env`.
- Run `uvicorn app.main:app --reload`.
- Open the browser at `127.0.0.1:8000`.
- Use `/docs` to test the JSON API interactively.

10.2 Testing Strategy

The supplied project contains a `pytest` suite with 26 tests. The test fixture replaces the Gemini client with a fake implementation and uses a temporary database. The tests cover page rendering, static assets, health checks, API validation, plan generation, database persistence, feedback updates, HTML form flow, escaping, missing users, Gemini errors, fallback behavior, empty responses and missing API keys.

During preparation of this documentation, the execution environment did not have the project's `google-genai` package installed and network access was unavailable, so the full test suite was not independently re-run here. The test coverage described above is taken from the supplied tests and README, not presented as a newly verified pass result.

11. API Reference

Endpoint	Request example	Response / behavior
POST <code>/generate-workout/gemini</code>	<code>{"goal":"weight loss","intensity":"Medium"}</code>	Returns model name and generated workout plan.
GET <code>/nutrition-tip</code>	<code>?goal=muscle%20gain</code>	Returns goal, tip and source (gemini-flash or fallback).
POST <code>/generate-plan</code>	User profile JSON	Generates and saves user + plan.
POST <code>/update-plan/7</code>	<code>{"feedback":"Add more core exercises"}</code>	Returns <code>updated_plan</code> .
DELETE <code>/users/7</code>	No body	Deletes user and associated plans.
GET <code>/health</code>	None	<code>{"status":"ok"}</code> .

The README also provides curl examples for the JSON endpoints and recommends `/docs` as the easiest interactive testing surface.

12. Database Design

Table	Column	Type / role
users	id	Integer primary key selected by user.
users	name	String(100).
users	age	Integer.
users	weight	Float, kilograms.
users	goal	String(200).
users	intensity	String(20).
users	schedule	Integer, default 7.
plans	id	Autoincrement integer primary key.
plans	user_id	Foreign key to users.id.
plans	original_plan	Text containing generated plan.
plans	updated_plan	Nullable Text containing latest revision.

The database helper opens short-lived SQLAlchemy sessions, commits changes explicitly and retrieves the latest plan for a user by descending plan ID. Deleting a user also deletes associated plans.

13. Security, Reliability and Error Handling

- API keys are read from environment variables and are not intended to be embedded in source code.
- The `.env.example` file contains a placeholder rather than a real key; the actual `.env` file should remain private and should not be committed.
- Gemini errors are mapped to clearer messages for invalid keys, forbidden requests, missing models, rate limits and server-side failures.
- Fatal errors such as invalid credentials are not retried through another model. Non-fatal model failures may use the configured Flash fallback.
- The web feedback route validates the User ID and verifies that a plan exists before calling Gemini.
- Jinja2 template rendering provides HTML escaping for user-supplied values.
- The admin view has no login according to the source comments; it should therefore remain local until proper authentication/authorization is implemented.
- The application is not medical advice. Generated exercise or nutrition content should be treated as general AI-generated wellness information and reviewed appropriately before use.

14. Screenshots and Implementation Evidence

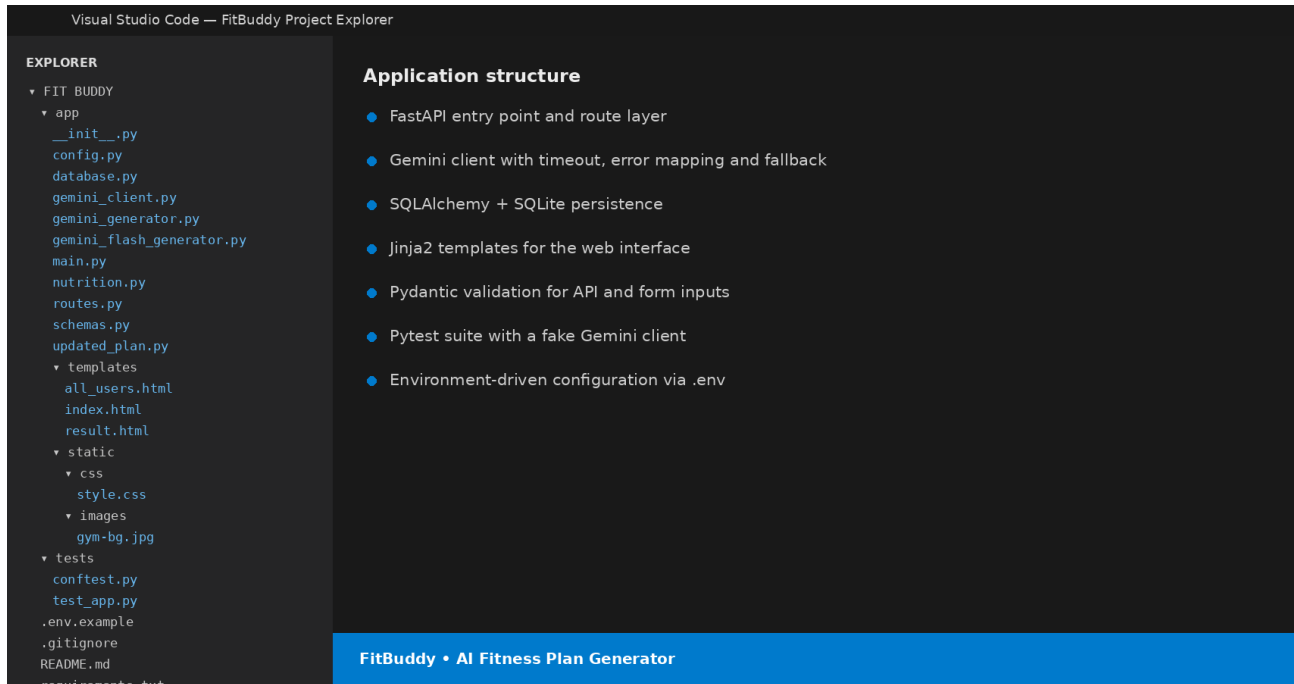


Figure 12. Project structure and module organization.

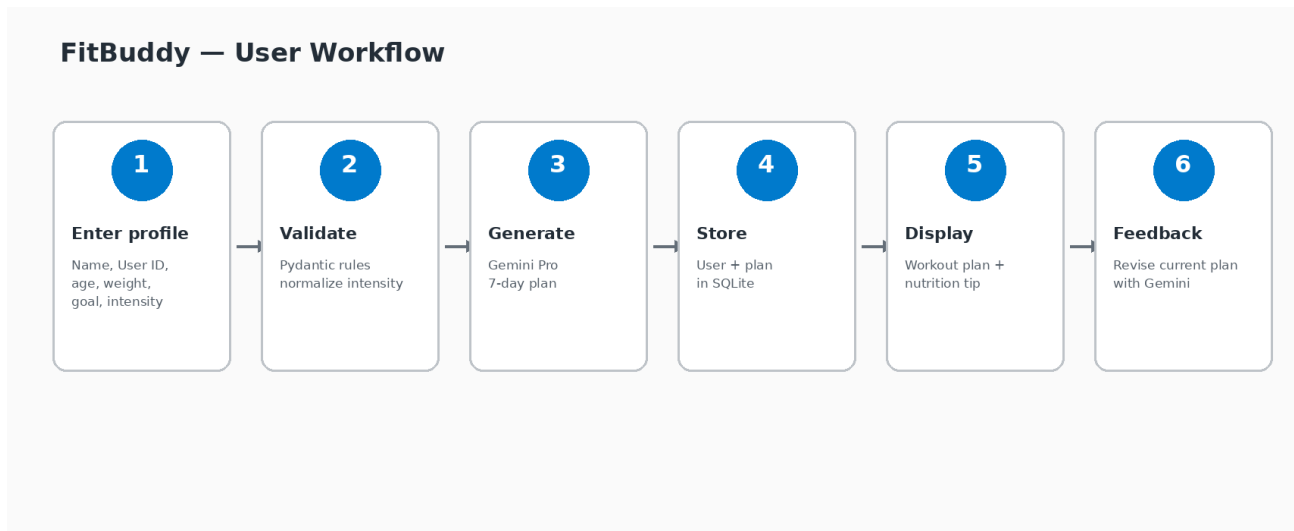
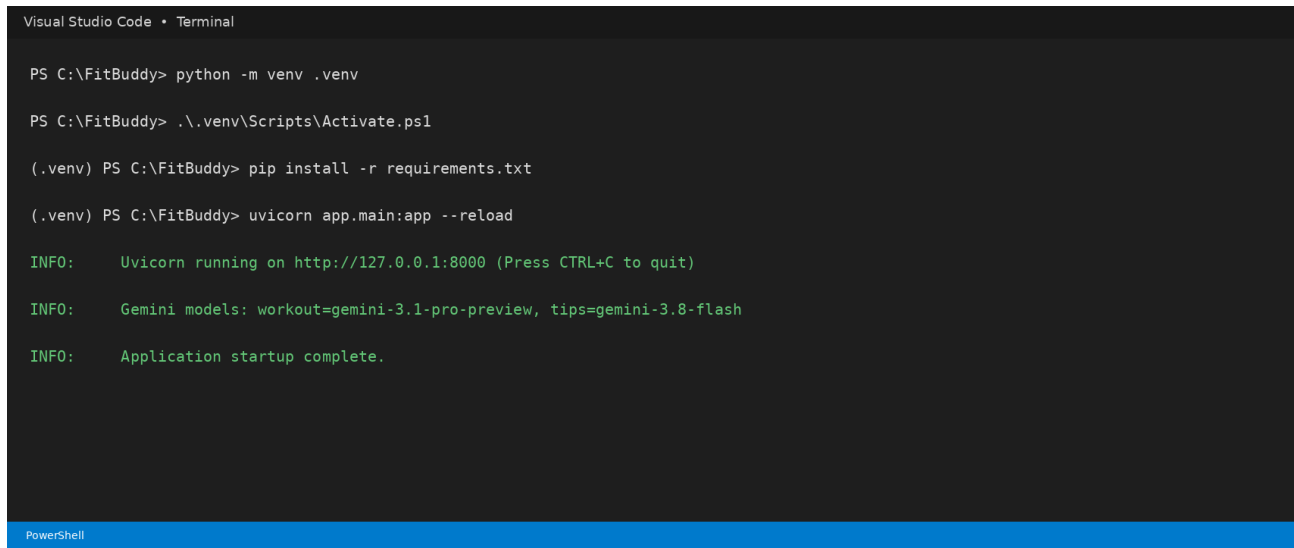


Figure 13. End-to-end user workflow.



```
Visual Studio Code • Terminal

PS C:\FitBuddy> python -m venv .venv

PS C:\FitBuddy> .\venv\Scripts\Activate.ps1

(.venv) PS C:\FitBuddy> pip install -r requirements.txt

(.venv) PS C:\FitBuddy> uvicorn app.main:app --reload

INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Gemini models: workout=gemini-3.1-pro-preview, tips=gemini-3.8-flash
INFO:      Application startup complete.
```

Figure 14. Local development commands and Uvicorn startup sequence.

The code screenshots in this report are generated from the actual files in the supplied FitBuddy project archive and presented in a VS Code-style visual frame for documentation readability. They are intended as implementation evidence rather than a claim that VS Code itself was captured from the user's machine.

15. Conclusion

FitBuddy demonstrates a complete AI-enabled web application workflow: user input is validated, a structured workout plan is generated through Gemini, a nutrition/recovery tip is produced through a fast AI path with an offline fallback, the result is persisted in SQLite, and user feedback can trigger a new revision of the current plan. The separation of routes, schemas, database helpers, Gemini services, templates and tests gives the project a clear modular structure.

The supplied reference documentation describes the same overall product idea and milestone progression, while the current project archive provides a more detailed implementation with configurable current Gemini model names, a centralized Google Gen AI client, JSON APIs, validation, deletion, fallback handling and automated tests. This report therefore preserves the reference document's professional project-report style while grounding technical details in the current codebase.